# LiveRecall: Networking Design and Evaluation

## 1. Architecture and Protocol

### 1.1 System Architecture

The supplied caption_source.py emits full-text revisions over WebSocket. A single asyncio server routes source and student connections by their first hello message. CaptionState validates updates and retains the latest active state; History stores recent finals. Every student connection has an independent bounded queue, sender and supervised session. The browser is served separately over HTTP and maintains its own display and local recall state. All lecture state is in memory, shared across source reconnects and lost on server restart.

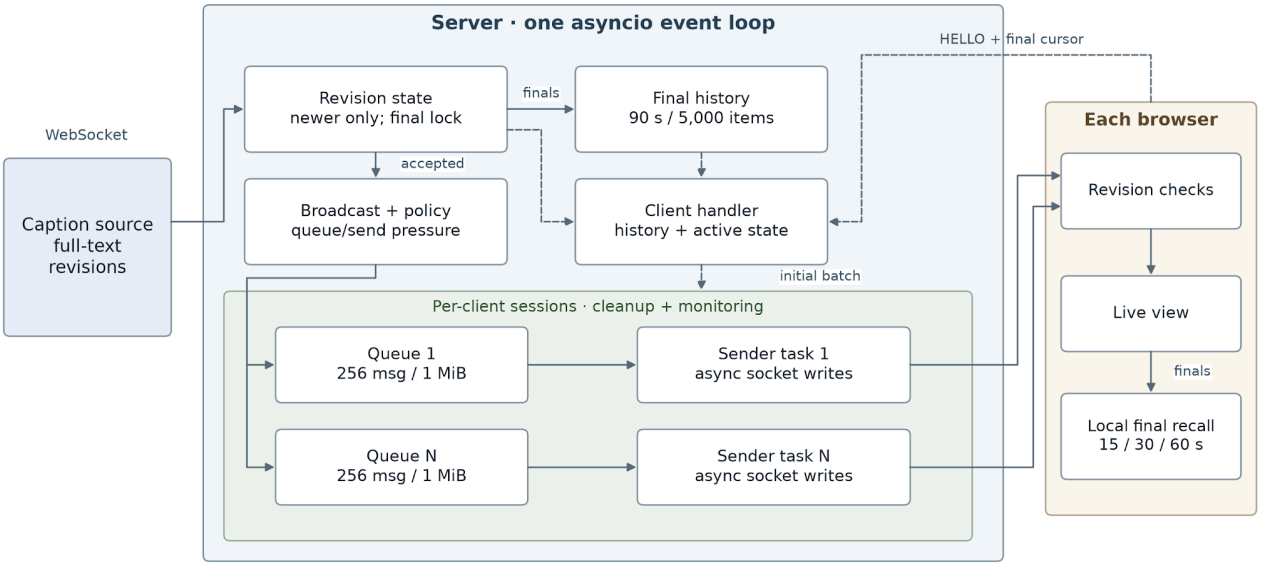



*Figure 1. Horizontal system architecture: validated state, final history and independent queues/senders. Dashed paths show recovery and client hello; each browser owns its display and recall state.*

### 1.2 Application Protocol

Messages are JSON text frames. SOURCE_HELLO identifies a source; CLIENT_HELLO supplies client_id, session, the last accepted final segment/revision and a requested history window. CAPTION_UPDATE carries session, segment, revision, source_ts, final and the complete text for that revision. HISTORY contains an oldest-first array of finalized CAPTION_UPDATE objects. Initial HISTORY precedes active snapshots and later live updates. SOURCE_DONE reports source completion for diagnostics. Hello/session fields route traffic; they do not provide authentication or independent multi-lecture state.

## 2. Revision and Recovery Semantics

### 2.1 Revision State and Bounded History

The server accepts an active segment only at a strictly newer revision and rejects all changes after finalization. The browser independently applies equivalent revision/final rules. Server state caps active segments at 1,000 and remembered final revisions at 10,000; a segment watermark rejects forgotten ranges. Full finals are retained separately for 90 seconds and at most 5,000 items. Retention uses server monotonic acceptance time, with pruning on insertion/query. Count and message-size limits can shorten the recoverable context under high load.

### 2.2 Local Recall and Browser Behavior

Live and recovered captions share applyCaption()/acceptUpdate(), so accepted revisions replace one segment line rather than adding duplicates. Provisional and final styling differs. The browser tracks at most 2,000 segments and renders at most 300 lines, with follow-live scrolling and a jump control.

Accepted finals enter a local rolling recall store, filtered by source timestamps: default 30 seconds, selectable 15/30/60 seconds. Normal recall reads this store without a server request. This timestamp comparison assumes synchronized source/browser clocks.

### 2.3 Reconnect and Late-Join Recovery

After closure, the browser retries after three seconds and reports its latest accepted final cursor; a fresh page sends 0/0. The client ID persists in sessionStorage, while caption state/cursor remain in page memory and reset on refresh. The server validates the cursor/window, caps requests at 60 seconds and selects missing finals within a 1 MiB message budget. It queues HISTORY and each active segment's latest provisional before registration, without an intervening await. Initial admission is all-or-nothing; oversized history can omit older finals. Recovery therefore depends on retention and capacity, not merely a successful reconnect.

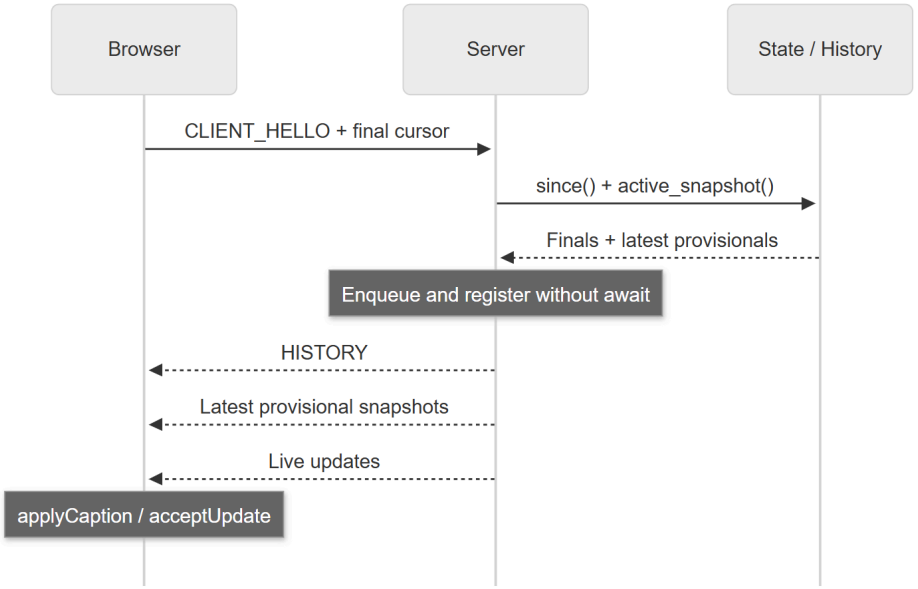



*Figure 2. Initial recovery is admitted before live registration; all browser updates share revision checks.*

## 3. Per-Client Buffering and Overload Handling

### 3.1 Queues, Senders and Session Lifecycle

Each connection owns a bounded FIFO, one asyncio sender and a supervised session. The producer enqueues without awaiting client socket writes, isolating normal receivers from blocked sends. Defaults cap pending payloads at 256 messages and 1 MiB; send and close attempts have 10 s and 3 s deadlines. Session teardown unregisters the connection, clears queued data, cancels and awaits child tasks, and removes monitoring/policy/eviction entries.

### 3.2 Monitoring and Overload Policy

Monitoring separates queued count/bytes/age, the in-flight payload, successful local sends and transport-buffer bytes. The current producer enables semantic coalescing when its per-client policy detects queue/send pressure. Degraded mode starts at 50% queue utilization or 0.5 s queue/send age; exit requires utilization ≤20% and ages ≤0.15 s. Existing finals are preserved; obsolete provisionals may be removed. Its older-segment dropping rule assumes the supplied source completes segments sequentially. Keepalive uses a 2 s ping interval and 5 s pong timeout; disconnected clients resume from their last accepted final cursor. This is timeout-driven recovery, not a precise measurement of displayed-caption lag.

## 4. Experimental Evaluation

### 4.1 Experimental Setup and Metrics

Each subsection identifies its own environment and workload. Runs were sequential, with a fresh server for each repetition and seed 3103. Values are medians of three per-run metrics, not pooled quantiles; T3 whiskers show the observed range. Live age is receive time minus source_ts and excludes HISTORY

items. Freshness counts accepted live updates within 500 ms. Final completeness counts unique finals from live and recovery against generated finals; fresh-final ratio applies the same age threshold to that denominator. Live coverage measures distinct received revisions. HISTORY-response latency measures request-to-response time, not caption age.

### 4.2 T1: Basic Correctness

- Environment: GB10; Ubuntu 24.04.4, ARM64, 20 logical CPUs; Python 3.12.3, websockets 16.1.1; commit 6dbd7da.
- Network: same-host loopback, no tc/netem impairment.
- Workload: 20 clients × 20 s × 3 runs; 22 final segments per run.
- Metrics: accepted live-caption P50/P95/P99 age, ≤500 ms freshness, final completeness including HISTORY, live revision coverage and connection errors.

Results: P50/P95/P99 = 1.27/1.68/2.41 ms; live freshness 100.00%, final completeness 100.00%, live revision coverage 100.00%. All three runs recorded zero duplicate/stale captions and zero connection errors. All generated revisions reached all simulated clients; this validates the basic public workload, not browser rendering or adversarial revision order.

### 4.3 T2: Public Scale

- Environment: GB10; Ubuntu 24.04.4, ARM64, 20 logical CPUs; Python 3.12.3, websockets 16.1.1; commit 6dbd7da.
- Network: same-host loopback, no tc/netem impairment.
- Workload: 100 clients × 40 s × 3 runs; 49 final segments per run.
- Metrics: accepted live-caption P50/P95/P99 age, ≤500 ms freshness, final completeness including HISTORY, live revision coverage and connection errors.

Results: P50/P95/P99 = 2.84/3.95/4.26 ms; live freshness 100.00%, final completeness 100.00%, live revision coverage 100.00%. All three runs recorded zero duplicate/stale captions and zero connection errors. Increasing the population from 20 to 100 raised tail latency, while freshness and completeness remained 100% in this loopback workload.

### 4.4 T3: Slow-Client Isolation and Before/After Comparison

- Environment: GB10; Ubuntu 24.04.4, ARM64, 20 logical CPUs; Python 3.12.3, websockets 16.1.1; commit 6dbd7da. Comparison: acef9a3 versus 6dbd7da; SHA-256 matched the unchanged public source, runner, clients and analyser.
- Network: same-host loopback, no tc/netem impairment.
- Workload: 100 clients × 60 s × 3 runs per version; 10 slow readers wait 500 ms per receive; 76 final segments/228 revisions per run.
- Metrics: accepted live-caption P50/P95/P99 age, ≤500 ms freshness, final completeness including HISTORY, live revision coverage and connection errors. Results are split into 90 normal and 10 slow clients.

*Table 1. Accepted live-caption latency and freshness; unique final completeness includes HISTORY recovery. Fresh means age ≤500 ms. Live coverage counts received revisions, not recovered finals.*

| Version / group | P50 (ms) | P95 (ms) | P99 (ms) | Fresh (%) | Final (%) | Live (%) |
|---|---|---|---|---|---|---|
| Before / normal | 2.93 | 5.52 | 6.00 | 100.00 | 100.00 | 100.00 |
| Before / slow | 14,712.05 | 28,076.50 | 29,127.81 | 1.69 | 51.32 | 51.75 |
| Current / normal | 2.86 | 4.43 | 5.91 | 100.00 | 100.00 | 100.00 |
| Current / slow | 6,960.80 | 13,356.43 | 13,805.85 | 2.65 | 100.00 | 49.56 |

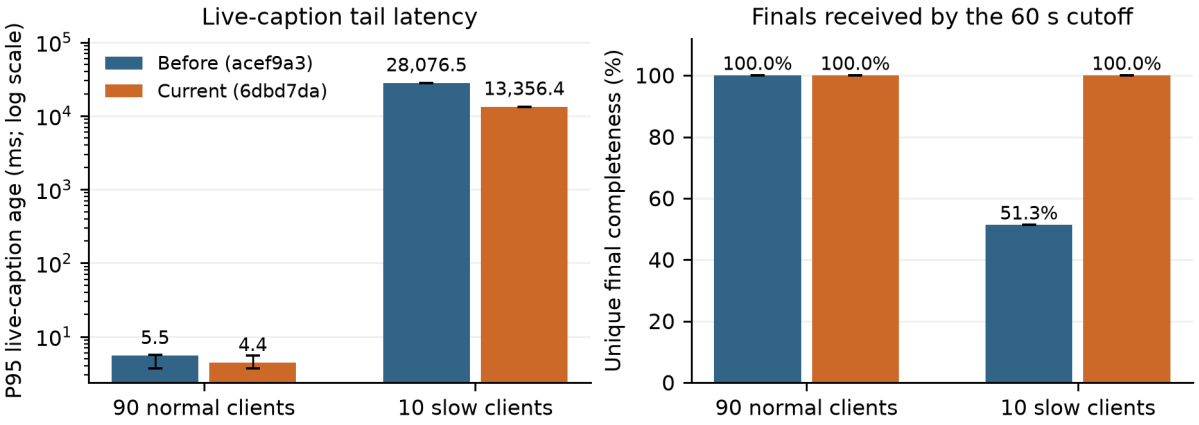



*Figure 3. Normal/slow T3 latency and final completeness before and after the full version change. Latency uses a log scale and excludes HISTORY items; the accepted-live sample populations differ between versions.*

Normal clients retained 100% live revision coverage and final completeness in both versions. Slow-client P95 live age changed from 28.08 s to 13.36 s, while final completeness at the cutoff changed from 51.32% to 100.00%. The current runs recorded 20 CONNECT_ERROR events and 380 recovered history items per run (medians); these reconnect costs must not be reported as zero errors. Both versions recorded zero duplicate/stale captions. Improved completeness is consistent with reconnect catch-up, but slow-client fresh-final ratio remained 1.32%: eventual correctness within this test did not provide real-time delivery. Median HISTORY-response P95 was 1.59 ms, which measures response time rather than recovered-caption freshness. This comparison changes the complete default version, not one isolated queue optimization.

### 4.5 T4: Reconnect and Recovery

- Environment: GB10; Ubuntu 24.04.4, ARM64, 20 logical CPUs; Python 3.12.3, websockets 16.1.1; commit 6dbd7da.
- Network: same-host loopback, no tc/netem impairment.
- Workload: 100 clients × 60 s × 3 runs; 20% disconnect at 15 s for 5 s; 76 finals per run.
- Metrics: accepted live-caption P50/P95/P99 age, ≤500 ms freshness, final completeness including HISTORY, live revision coverage and connection errors.

Results: P50/P95/P99 = 3.15/5.26/5.73 ms; live freshness 100.00%, final completeness 100.00%, live revision coverage 98.25%. All three runs recorded zero duplicate/stale captions and zero connection errors. Each run recovered 140 history items through 20 reconnects. HISTORY-response P95 was 1.87 ms; fresh-final ratio was 98.42%. Missing offline provisional revisions explain coverage below 100%; recovery restores finals rather than every intermediate revision.

### 4.6 T5: Network Impairment

- Environment: GB10; Ubuntu 24.04.4, ARM64, 20 logical CPUs; Python 3.12.3, websockets 16.1.1; commit 6dbd7da.
- Network: isolated Linux network namespace; supplied delay_loss.sh on lo (120 ms ±30 ms normal jitter, 3% loss). The source-to-server and server-to-client paths both traverse this interface; these are not end-to-end one-way delay values. Profile reset after all runs.
- Workload: 100 clients × 40 s × 3 runs; 49 finals per run.
- Metrics: accepted live-caption P50/P95/P99 age, ≤500 ms freshness, final completeness including HISTORY, live revision coverage and connection errors.

Results: P50/P95/P99 = 255.91/691.05/1002.58 ms; live freshness 89.12%, final completeness 100.00%, live revision coverage 99.96%. All three runs recorded zero duplicate/stale captions. All three runs recorded zero connection errors. Fresh-final ratio was 86.86%. TCP reliability preserved eventual final delivery here, but retransmission, ordered delivery and jitter increased tail age beyond the freshness target. This experiment does not measure a bandwidth-halving scenario. Appendix A compares the public workloads and shows the three T5 receive-age distributions.

### 4.7 Server Memory Observations

- Environment: GB10; Ubuntu 24.04.4, ARM64, 20 logical CPUs; Python 3.12.3, websockets 16.1.1; commit 6dbd7da. Same loopback environment as T3, without impairment.
- Method/metrics: server-child VmRSS sampled every 0.5 s, tracked by PID; compare three T3 runs/version and one additional 300 s current run with the same 90/10 client mix. Report peak, early/late median and post-60 s slope.

An external observer sampled only each runner's server child VmRSS every 0.5 s, recording PID changes between runs. Peak T3 RSS ranged from 31.93–31.99 MiB before and 33.38–33.42 MiB currently. A separate 300 s current-version run with the same client mix peaked at 36.65 MiB; median RSS was 32.15 MiB during seconds 10–30 and 36.65 MiB in its final 30 s. The upward trend was approximately 0.76 MiB/min after 60 s, so these data do not establish a plateau, leak freedom or long-term boundedness.

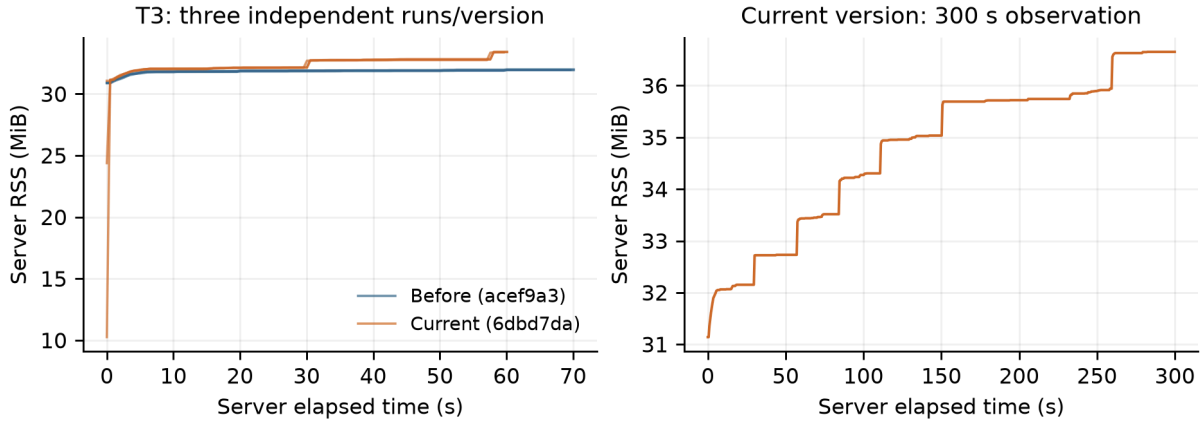



*Figure 4. Per-server RSS for all three T3 repetitions and the separate 300 s observation. Sampling includes startup/teardown. RSS excludes clients and kernel socket buffers; the single long run provides no confidence interval.*

## 5. Failures, Trade-offs and Limitations

Bounded queues protect resource usage but unresolved overload can still evict clients. Frequent reconnects trade catch-up completeness against disruption and additional history traffic. Current T3 slow-reader freshness remains poor. Initial HISTORY can omit older finals when its byte budget is exceeded, and arbitrary interleaved active segments require separate validation. Measurements neither isolate which optimization caused the change nor establish behavior under bandwidth shaping, larger populations or long-term memory growth. The 300 s run is supplementary, not a repeated formal T3 result. Ubuntu unit tests passed (131 tests), but these observations do not establish full hidden-test correctness.

## 6. Future Work and FYP Direction

Delivery-focused next steps are complete bounded/chunked final recovery, explicit validation of interleaved active segments, and bandwidth shaping and repeated loss experiments with receiver-aware lag measurements. Longer connection-churn studies should include process and kernel memory.

## References

[1] CS3103 Assignment 4: LiveRecall handout, updated 17 October 2026, sections 8, 10 and 12.

[2] Group repository snapshots acef9a301432cea5936c1303ce1799b8905d904a and 6dbd7daa03c729f03f78a944be0871d31b92cc63. Commands, per-run CSV/JSON, RSS traces, source hashes and plotting scripts: experiments/delivery_20261103/.

# Appendix A. Additional Experimental Figures

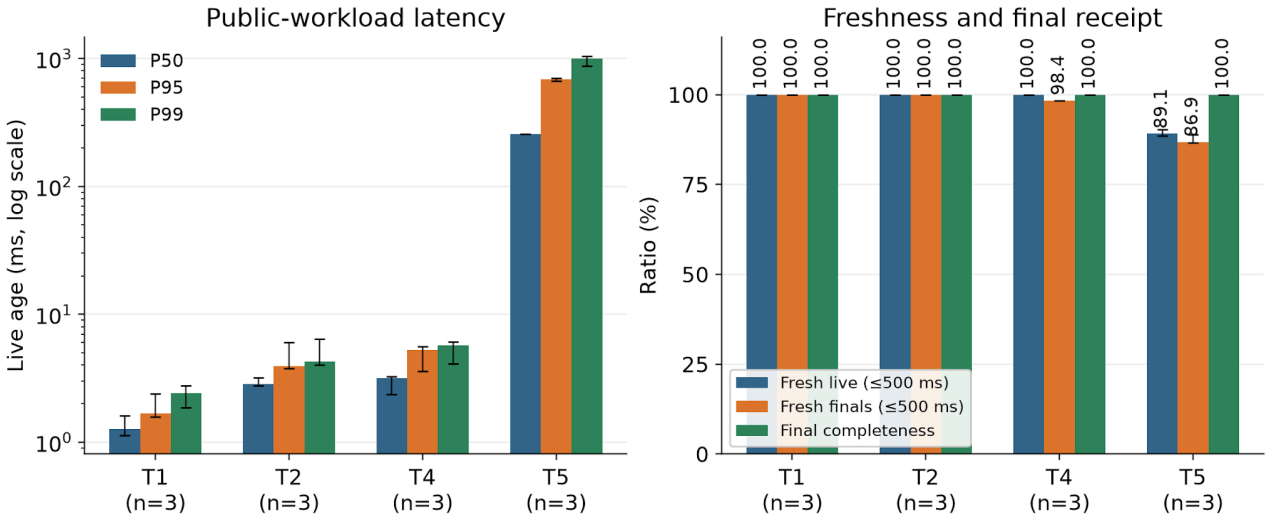



*Figure 5. T1/T2/T4/T5 medians of three per-run metrics; whiskers show observed min/max, not confidence intervals. Latency excludes HISTORY; finals include recovery. T3 is shown separately in Figure 3. Workloads differ, so this is not a controlled scaling curve.*

Unshaped T1/T2/T4 retain 100% live freshness, while T5 delay/loss increases tail age and lowers freshness despite 100% final completeness. T4's offline period reduces fresh-final ratio because recovered finals are older than the 500 ms target.

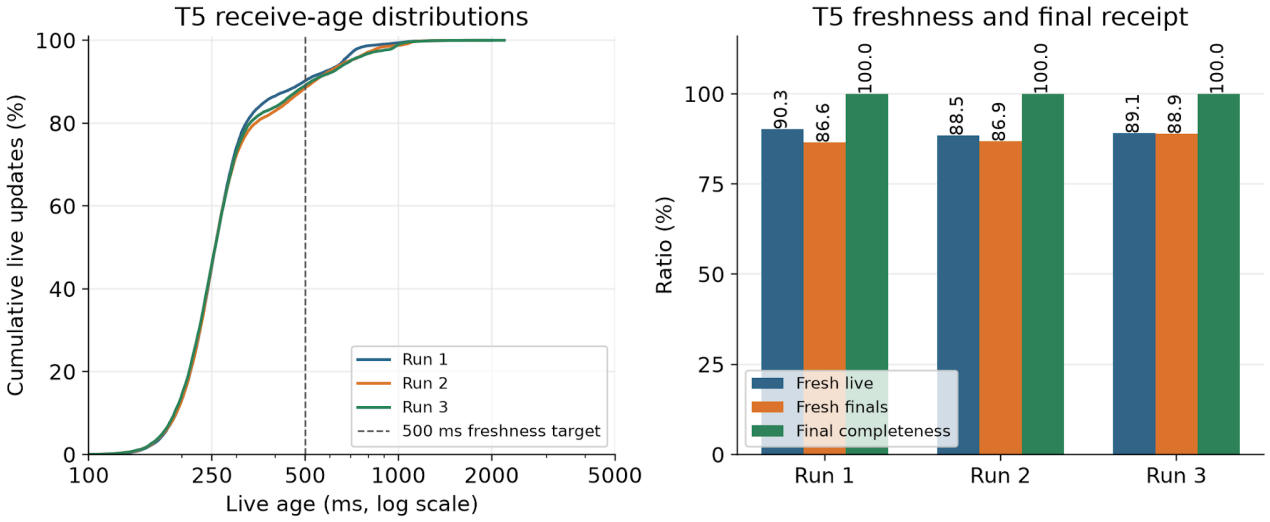



*Figure 6. T5 per-run empirical cumulative live-age distributions and freshness/final completeness, using the accepted three-run batch. Each curve includes all accepted live updates from one run; the dashed line marks 500 ms. Latency uses a log scale.*

All three runs converge to 100% final completeness, while only about 89% of live updates arrive within 500 ms. The receive-age tails show that eventual reliability and timely delivery are separate properties. The fixed source seed does not fix netem's random loss/jitter.